

OBSERVE - DESIGN PATTERN

Why choose an Observe design pattern?

The **Observer pattern** is necessary in a *Library Management System* because the system contains many situations where a change in one object must automatically trigger actions in multiple other components. Without Observer, the system becomes tightly coupled, harder to maintain, and difficult to extend.

Problem: A book is currently borrowed. Many students want it. When the book becomes available:

- notify waiting students
- update UI
- update inventory status

Without Observer:

- library system manually contacts everyone

This thing is messy and hard to maintain.

Solution: The **Observer Pattern** solves this problem by allowing the Book object (Subject) to automatically notify all subscribed students or services (Observers) whenever its availability changes.

Benefits of Using Observer Pattern

- **Low Coupling:** the book does not need to know details about each student or service
- **Easy Maintenance:** changes can be made without affecting the whole system
- **Scalability:** new observers (email service, mobile app, UI updates) can be added easily
- **Automatic Updates:** all subscribed users receive real-time notifications

Class Diagram:

